

Uflex: A Lexical Analyzer Generator for Unicon

Clinton Jeffery and Susie Jeffery

Unicon Technical Report: 2b

2026-04-03

Abstract

Uflex is a lexical analyzer generator for Unicon. It strongly resembles the original Lex language, and explicitly supports development of lexical specifications that can be shared with a tool for Java called JFlex. This report describes Uflex version 0.9.

Unicon Project

<http://unicon.org>

University of Idaho

Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

Parsing is a task in a compiler or interpreter that analyzes source code input and reports on its compliance and syntax structure. *Scanning*, or lexical analysis, provides the parser's input in the form of a sequence of lexical items (words, tokens, or lexemes) from the input source code.

Unicon has several string processing facilities that can be used for lexical analysis. These include string scanning, SNOBOL-style patterns with regular expressions as their literals, and various Icon Program Library modules. However, for language processing tasks such as writing compilers and interpreters, declarative specifications of lexical and syntax rules give better maintainability and modifiability than handwritten code, generally at the cost of some performance.

Uflex is a tool that generates a scanner. Uflex stands for Unicon's Friendly Lexical Analyzer. It behaves similar to the UNIX `lex(1)` [Lesk75] tool, generating Unicon code as its output. Uflex is influenced by Flex and Jflex, the modern open source descendants of Lex for C and Java.

Uflex takes an input specification for the desired lexical analysis that consists of a number of *regular expressions*. Most languages' lexical structure can be specified using this notation. Uflex's regular expression notation is summarized in Table 1.

Operator	Description
<code>a</code>	ordinary symbols match themselves
<code>.</code>	a dot matches any one character other than newline
<code>e\ ₁ \ e\ ₂</code>	a bar matches either the expression on its left or its right
<code>e\ ₁ e\ ₂</code>	two expressions match the left followed by the right
<code>e\ *</code>	star matches zero or more occurrences of an expression
<code>[\ abc \]</code>	square brackets match any one character within the brackets
<code>e\ +</code>	plus matches one or more occurrences of an expression
<code>e\ ?</code>	

matches zero or one occurrences of the preceding expression

"..."

matches characters in quotes literally

(e)

parentheses override operator precedence

(?# comment)

a comment

Table 1: Regular Expressions in Uflex.

The unary postfix operators `\*` and `+` and `?` are higher precedence than the binary operators, and alternation is lower precedence than concatenation. In Table 1, the square bracketed character set notation has several extensions, including ranges such as `[a-z]`, negation `[^abc]`, and a set of predefined named csets. The predefined named csets correspond to C's `<ctype.h>` macros and appear as square bracketed items within a cset definition, for example `[[:alnum:]]` denotes any one alphanumeric character. Do not blame us for this interesting notation, it comes from Flex.

This document assumes you are familiar with regular expressions; if not, you may also wish to read *flex & bison* [Levine09]. Uflex is usually used with `lyacc`, a parser generator tool documented in [Pereda18]. Uflex and `lyacc` are additionally described in *Programming with Unicon* [Jeffery03].

2 2. Command Line Invocation

Uflex is invoked like this:

```
uflex [options] lexfile[.l]
```

The lex specification file normally ends in `.l`. This suffix is automatically added if it is omitted and adding `.l` will result in a valid filename.

The uflex options are as follows:

option

meaning

-automaton

write out finite automaton of uflex regular expressions

-nfa

This option causes Uflex to write out a non-deterministic finite automaton, which is slower but sometimes less buggy than the default deterministic finite automaton.

-nopp
skip the macro preprocessor step
-o \ *“file”*
write output to file (default is flex-style: `basename.icn`)
-token
write out tokens
-tree
write out parse tree of uflex regular expressions
-version
write the uflex version and stop

3 3. Uflex Program Structure

The Uflex tool takes a lexical specification and produces a lexical analyzer that corresponds to that specification. The specification consists of a list of regular expressions, with auxiliary code fragments, variables, and helper functions. The resulting generated analyzer is in the form of a procedure named `yylex()`.

All Uflex programs consist of a file with an extension of `.l` that contains three sections, separated by lines consisting of two percent signs. The three sections are the definitions section or header, the rules section, and the procedures section.

3.0.1 3.1 Header Section

The definitions section has two kinds of components: macros and code fragments. *Macros* define shorthand for regular expressions that will be used in the next section. A macro is given by a line starting with the macro name in straight alphanumeric form, followed by a space, followed by a regular expression. Beware the simple macro syntax. *Code fragments* enclosed by `%{` and `%}` are copied verbatim to the generated lexical analyzer. Usually they will be used to declare global variables and types or include files.

3.0.2 3.2 Rules Section

The rules section contains the actual regular expressions that specify the lexical analysis that is to be performed. Each regular expression may be followed by an optional semantic action enclosed in curly brackets, which

is a segment of Unicon code that will be executed whenever that regular expression is matched.

Here are some extended flex regular expressions, some of which are supported in the uflex rules section. See the appendix for some other flex features that uflex does not yet support.

```
[a-z]{+}[aeiou]
character set union
[a-z]{-}[aeiou]
character set difference/subtraction
\\0
NUL
\\123
octals
\\xHH
hexadecimals
[[:...:]]
predefined csets
[:alnum:]
&letters++&digits
[:cntrl:]
[\\1-\\32]
[:lower:]
&lcase
[:space:]
[ \\t\\f\\v\\n\\r]
[:alpha:]
a-zA-Z
[:digit:]
0-9
[:print:]
[\\24-\\176]
[:upper:]
&ucase
[:blank:]
[ \\t]
[:graph:]
[:print:]{-}[ ]
[:punct:]
```

```
[:graph:]{-}([0-9a-zA-Z])
[:xdigit:]
[0-9a-fA-F]
```

3.0.3 3.3 Procedures Section

The procedures section is also copied out verbatim into the generated lexical analyzer. It may include a `main()` procedure for standalone Uflex programs, although more frequently it may include helper functions called from within the semantic actions in the rules section. The following public interface is used to communicate with Uflex-generated lexical analyzers when they are called from other modules.

The generated `yyllex()` function and its return value constitute the primary interface between the lexical analyzer and the rest of the program. `yyllex()` returns a -1 if it consumes the entire input; returning different integer values from within semantic actions in the rules section allows `yyllex()` to break the input up into multiple chunks of 1+ characters (called tokens), and to identify different kinds of tokens using different integer codes. In addition to the return value, the generated lexical analyzer also makes use of several global variables. The names and meanings of these are summarized in Table 2.

Variable Name	Description
<code>yyin</code>	File from which characters will be read; default: <code>&input</code>
<code>yytext</code>	String of characters matched by a regular expression
<code>yyleng</code>	Length of <code>yytext</code>
<code>yychar</code>	Integer category of the most recent token
<code>yylval</code>	Lexical values (often an object or record) of the most recent token

Table 2: Uflex global variables.

4 4. Examples

The best way to explore the capabilities of uflex is to look at examples. This section includes a couple of traditional toy standalone examples. The uflex distribution includes a test/ directory containing additional examples that handle toy subsets of C, C++, Java, Python, Go, Kotlin, and Rust. The evaluation section below includes a discussion of a Uflex specification written for a Unicon lexical analyzer. See [Jeffery24] for additional discussion of the Uflex specification for a subset of Java, which also compiles and runs unmodified under Jflex.

4.0.1 4.1 A Word Count Program

There is a UNIX program called wc, short for word count, that counts the number of lines, words, and characters in a file. This example demonstrates how to build such a program using Uflex. A short, albeit simplistic, definition of a word is any sequence of non-white space characters, where white space characters are blanks and tabs. See Listing 1 for a Uflex program that operates like wc.

```
ws [
nonws [^          t          n]
%{
global cc, wc, lc
%}
%%
{nonws}+ { cc += yyleng; wc += 1 }
{ws}+ { cc += yyleng }
      n { lc += 1; cc += 1 }

%%
procedure main()
cc := wc := lc := 0
yylex()
write(right(lc, 8), right(wc, 8), right(cc, 8))
end
```

Listing 1. wc using uflex.

In the word count program, the definitions section consists of two definitions, one for white space characters (**ws**) and one for non-white space characters (**nonws**). These definitions are followed by code to declare three global variables: **cc**, **wc**, and **lc**. These are the counters for characters, words, and lines, respectively. The rules section in this example contains three rules. White space, words, and newlines each have a rule that matches and counts their occurrences. The procedure section has one procedure, **main()**. It calls the lexical analyzer and then prints out the counted values.

4.0.2 4.2: A Lexical Analyzer for a Desktop Calculator

The previous example demonstrates using Uflex to create standalone programs. However, **yylex()** is typically called from a parser. The **yylex()** function can be used to produce a sequence of words so that a parser such as that generated by the **iyacc** program can combine those words into sentences. Thus it makes sense to study how **uflex** is used in this context. One obvious difference is that in the earlier example, **yylex()** was only called once to process the entire file. In contrast, when a parser uses **yylex()**, it calls the analyzer repeatedly, and **yylex()** returns with each word that it finds. This will be demonstrated in the example that follows.

A calculator program is simple enough to understand in one sitting and complex enough to get a sense of how to use Uflex with its parser generator counterpart: **Iyacc**. In a general desktop calculator program, the user types in complex formulas, which the calculator evaluates and then prints the result. The generated lexical analyzer must recognize the words of this language, which will be handled by the parser. In this case the words are numbers, math operators, and variable names.

A number is one or more digits, followed by an optional decimal point and one or more digits. In regular expressions, this may be written as:

`[0-9]+(.[0-9]+)?`

The math operators are simple words composed of one character. Variable names can be any combination of letters, digits, and underscores. So as not to confuse them with numbers, refine the definition by making sure that the variables do not begin with a number. This definition of variable names corresponds to the following regular expression:


```
[a-zA-Z_][a-zA-Z0-9_]*
```

Recall that there are three sections to every Uflex program: a definitions section, a rules section, and a procedures section. The Uflex program for matching the words of a calculator is given in Listing 2.

```
%{
# y_tab.icn contains the integer codes for representing the
# terminal symbols NAME, NUMBER, and ASSIGNMENT.
#include y_tab.icn
%}
letter [a-zA-Z_]
digiletter [a-zA-Z0-9_]

%%
{letter}{digiletter}* { yylval := yytext; return NAME }
[0-9]+(.[0-9]+)? { yylval := numeric(yytext); return NUMBER }
n {
    return 0 # logical end-of-file
}
":=" { return ASSIGNMENT }
[
    t]+ { # ignore white space }
. { return ord(yytext) }
%%
```

Listing 2. Uflex program for recognizing the lexical elements of a calculator.

The definitions section has both a component that is copied directly to the generated lexical analyzer as well as a set of macros. The first rule matches variable names; the second rule matches numbers. The third rule returns 0 to indicate to the parser that it should evaluate the expression. The fourth rule lets the parser know that there was an assignment operator, and the fifth is used to ignore white space. The last rule matches everything else including the other mathematical operators. The character's numeric code (e.g. ASCII) is returned directly to the parser.

`yylval` is used to store the result whenever we match either a variable name or a number. This way when the lexical analyzer returns the integer

code for name or number, the parser knows to look in `yylval` for the actual name or number that was matched. Since Unicon allows variables to hold any type of value there is no need for a complicated construct to handle the fact that different tokens have different types of lexical values.

Notice that the matches that are allowed by this set of regular expressions are somewhat ambiguous. For example, `count10` may match a variable name and then an integer, or one variable name. The Uflex tool is designed to match the longest substring of input that can match the regular expression. So `count10` would be matched as one word, which is a variable in this case. In the case where two different expression match the same number of characters, the first rule listed in the specification will be used.

4.0.3 4.3: A Lexical Analyzer for Unicon

A lexical analyzer for Unicon is provided in `uflex's test/` directory in a file named `uniflex.l`. It produces output that is identical to Unicon's handwritten lexical analyzer `unilex.icn` on the largest source file in the Unicon distribution, `ipl/gprogs/gpxtest.icn`. So, it is pretty close and can give the reader a feel for the extent to which `uflex` is ready for real-world uses at this point. There is various fancy stuff to support semi-colon insertion, etc.

```
%{
# A Uflex Unicon lexer

#include "ytab_h.icn"
#define Beginner 1
#define Ender 2
#define Newline 4
record token(tok, s, line, column, filename)
%}

O [0-7]
D [0-9]
L [[:alpha:]]_
H [[:xdigit:]]
R [[:alnum:]]
FS [fFlL]
IS [uUlL]
```

```

W [          t          v]
idchars [[:alnum:]]_
global yylineno, yycolno, yyfilename, tokflags, lastid, buffer, lastchar

%%

"abstract" { return ABSTRACT }
"break"    { tokflags += Beginner+Ender; return BREAK }
"by"       { return BY }
"case"     { tokflags += Beginner; return CASE }
"class"    { return CLASS }
"create"   { tokflags += Beginner; return CREATE }
"critical" { tokflags += Beginner; return CRITICAL }
"default"  { tokflags += Beginner; return DEFAULT }
"do"       { return DO }
"else"     { return ELSE }
"end"      { tokflags += Beginner; return END }
"every"    { tokflags += Beginner; return EVERY }
"fail"     { tokflags += Beginner+Ender; return FAIL }
"global"   { return GLOBAL }
"if"       { tokflags += Beginner; return IF }
"import"   { return IMPORT }
"initial"  { tokflags += Beginner; return iconINITIAL }
"initially" { tokflags += Ender; return INITIALLY }
"invocable" { return INVOCABLE }
"link"     { return LINK }
"local"    { tokflags += Beginner; return LOCAL }
"method"   { return METHOD }
"next"     { tokflags += Beginner+Ender; return NEXT }
"not"      { tokflags += Beginner; return NOT }
"of"       { return OF }
"package"  { return PACKAGE }
"procedure" { return PROCEDURE }
"record"   { return RECORD }
"repeat"   { tokflags += Beginner; return REPEAT }
"return"   { tokflags += Beginner+Ender; return RETURN }
"static"   { tokflags += Beginner; return STATIC }
"suspend"  { tokflags += Beginner+Ender; return SUSPEND }
"then"     { return THEN }
"thread"   { tokflags += Beginner; return THREAD }

```

```

"to"          { return TO }
"until"       { tokflags += Beginner; return UNTIL }
"while"       { tokflags += Beginner; return WHILE }
{L}{idchars}* { tokflags += Beginner+Ende; return IDENT }

'[^'      n]* ' { tokflags += Beginner + Ende; return CSETLIT }
"([^\n]|"_n"|" "[n])*" { tokflags += Beginner + Ende; mls(); return STRINGLIT }
"!"          { tokflags += Beginner; return BANG }
"%"          { return MOD }
"%:="        { return AUGMOD }
"&"          { tokflags += Beginner; return AND }
"&&"         { return PAND }
"&:="        { return AUGAND }
"*"          { tokflags += Beginner; return STAR }
"**"         { tokflags += Beginner; return INTER }
"**:="       { return AUGINTER }
*:="         { return AUGSTAR }
"+"          { tokflags += Beginner; return PLUS }
++"          { tokflags += Beginner; return UNION }
++:="        { return AUGUNION }
+:="         { return PCOLON }
+:="         { return AUGPLUS }
-"           { tokflags += Beginner; return MINUS }
->"          { return PASSNONMATCH }
--"          { tokflags += Beginner; return DIFF }
--:="        { return AUGDIFF }
-:"          { return MCOLON }
-:="         { return AUGMINUS }
"."          { tokflags += Beginner; return DOT }
".$"         { return PSETCUR }
".>"         {
    # missing logic here if next_gt_is_ende == 1 then ...
    return PSETCUR
}
".|" { return POR }
"."[0-9]+ { tokflags += Beginner + Ende; return REALLIT }
"/"        { tokflags += Beginner; return SLASH }
"/:="      { return AUGSLASH }
":"        { return COLON }
::="       { tokflags += Beginner; return COLONCOLON }

```

```

":="      { return ASSIGN }
":="      { return SWAP }
"<"       { return NMLT }
"<="      { return NMLE }
"<:="     { return AUGNMLE }
"<<"      { return SLT }
"<<="     { return SLE }
"<<:="    { return AUGSLE }
"<<:="    { return AUGSLT }
"<<@" { tokflags += Beginner + Ender; return RCVBK }
"<-" { return REVASSIGN }
"<->" { return REVSWAP }
"<:" { return AUGNMLT }
"<@" { tokflags += Beginner + Ender; return RCV }
"=" { tokflags += Beginner; return NMEQ }
">=" { return PIMDASSN }
"=:=" { return AUGNMEQ }
"==" { tokflags += Beginner; return SEQ }
"==:" { return AUGSEQ }
"===" { tokflags += Beginner; return EQUIV }
"===:" { return AUGEQUIV }
">" {
    if next_gt_is_ender then {
        tokflags += Ender
        next_gt_is_ender := &null
    }
    return NMGT
}
">=" { return NMGE }
">=:=" { return AUGNMGE }
">>" { return SGT }
">>=" { return SGE }
">>:=" { return AUGSGE }
">>:=" { return AUGSGT }
">:" { return AUGNMGT }
"?" { tokflags += Beginner; return QMARK }
"??" { return PMATCH }
"?:" { return AUGQMARK }
"@ " { tokflags += Beginner; return AT }
"@>>" { tokflags += Beginner + Ender; return SNDBK }

```

```

"@>" { tokflags += Beginner + Ender; return SND }
"@:=" { return AUGAT }
"      " { tokflags += Beginner; return BACKSLASH }
"^" { tokflags += Beginner; return CARET }
"^:=" { return AUGCARET }
"|" { tokflags += Beginner; return BAR }
"||" { tokflags += Beginner; return CONCAT }
"|||" { tokflags += Beginner; return LCONCAT }
"||:=" { return AUGCONCAT }
"||||=" { return AUGLCONCAT }
"~" { tokflags += Beginner; return TILDE }
"~=" { tokflags += Beginner; return NMNE }
"~:=" { return AUGNMNE }
"~==" { tokflags += Beginner; return SNE }
"~==:" { return AUGSNE }
"~===" { tokflags += Beginner; return NEQUIV }
"~===:" { return AUGNEQUIV }
"(" { tokflags += Beginner; return LPAREN }
")" { tokflags += Ender; return RPAREN }
 "[" { tokflags += Beginner; return LBRACK }
 "]" { tokflags += Ender; return RBRACK }
 "{" { tokflags += Beginner; return LBRACE }
 "}" { tokflags += Ender; return RBRACE }
 "," { return COMMA }
 ";" { return SEMICOL }
"$(" { tokflags += Beginner; return LBRACE }
"$)" { tokflags += Ender; return RBRACE }
"$<" { tokflags += Beginner; return LBRACK }
"$>" { tokflags += Ender; return RBRACK }
"$ $" { return PIMDASSN }
"$" { return DOLLAR }
"^[^']*" { tokflags += Ender; return PUNEVAL }
"^[^']*" { tokflags += Ender; return PUNEVAL }
{D}+ { tokflags += Beginner+Ender; return INTLIT }
{D}+[rR]{R}+ { tokflags += Beginner+Ender; return INTLIT }
{D}+[kK] { tokflags += Beginner+Ender; yytext:=string(yytext[1:-1]*1024); return INTLIT }
{D}+[mM] { tokflags += Beginner+Ender; yytext:=string(yytext[1:-1]*1024^2); return INTLIT }
{D}+[gG] { tokflags += Beginner+Ender; yytext:=string(yytext[1:-1]*1024^3); return INTLIT }
{D}+[tT] { tokflags += Beginner+Ender; yytext:=string(yytext[1:-1]*1024^4); return INTLIT }
{D}+[pP] { tokflags += Beginner+Ender; yytext:=string(yytext[1:-1]*1024^5); return INTLIT }

```

```

{D}+[dD]{D}+ { tokflags += Beginner+Ender; return INTLIT }
{D}+((([eE][+-]{D}+)?) { tokflags += Beginner+Ender; return REALLIT }
{D}+ .{D}+ { tokflags += Beginner+Ender; return REALLIT }
{D}+.{D}*((([eE][+-]{D}+)?) { tokflags += Beginner+Ender; return REALLIT }
#line "[0-9]+" "[^" "*" ".* {
    yytext ? { move(6)
        yylineno := integer(tab(many(&digits)));
        if =" "" & (new_filename := tab(find(" ""))) then
            yyfilename := new_filename
        }
    }
}
"#{".* { }
"
    n" { yylineno += 1; yycolno := 1;
        if tokflags < Newline then tokflags += Newline }
" " { yycolno += 1 }
[
    v
    014] { }
" t" { yycolno += 1; while(yycolno-1)%8~=0 do yycolno += 1 }

%%

procedure lex_error()
    yyerror("token not recognized")
end

procedure uni_error(s)
    /errors := 0
    /s := "token not recognized"
write("uni_error calls yyerror ", image(s))
    yyerror(s)
    errors += 1
end

# implement semi-colon insert here, swap yylex:=:yylex2
procedure yylex2()
    static saved_tok, saved_yytext, seen_eof
    local rv
    if
        seen_eof then return 0
    if
        saved_tok then {
        rv := saved_tok
        saved_tok := &null

```

```

    yytext := saved_yytext
    yylval := yytoken := token(rv, yytext, yylineno, yycolno, yyfilename)
    if
        debuglex then
        write("yylex() : ", tokenstr(rv), "    t", image(yytext))
    return rv
}
if /ender := iand(tokflags, Ender) then
    tokflags := 0
if not (rv := yylex2()) then {
    yytext := ""
    if
        debuglex then
        write("yylex() : EOFX")
        seen_eof := 1
        return EOFX
}
if ender~=0 & iand(tokflags, Beginner)~=0 & iand(tokflags, Newline)~=0 then {
    saved_tok := rv
    saved_yytext := yytext
    yytext := ";"
    rv := SEMICOL
}
yylval := yytoken := token(rv, yytext, yylineno, yycolno, yyfilename)
if
    debuglex then
    write("yylex() : ", tokenstr(rv), "    t", image(yytext))
return rv
end

# handle multi-line string, eating underscore-newline pairs and the leading
# spaces of the next line
procedure mls()
    outs := ""
    yytext ? {
        while outs ||:= tab(find("_"          n")) do {
            move(2); tab(many('          t'))
        }
        outs ||:= tab(0)
    }
    yytext := outs
    yyleng := *yytext

```



```
end
```

5 5. Evaluation

While still being refined at this point, `uflex 0.8` is now correct enough for use on production tools. Flaws and limitations are still being repaired or documented. The Uflex test suite, in a subdirectory named `test/`, allows you to run a range of examples that validate working functionality and identify areas of deficiency. It could benefit from more examples and larger test inputs.

One basic question about any lexical analyzer whether its speed is acceptable. Historically, the original UNIX `lex(1)` was famous for being slow. If a generated lexical analyzer is too much slower than a handwritten lexical analyzer, then the tool is unattractive despite its advantage in terms of improved maintainability. Performance mattered a lot more when compilers were glacial and every bit of speed counted. At that time, if you could double the speed of your lexical analyzer and make your whole compiler 10% faster, it made a huge difference in programmer productivity. Performance is less of a concern at this point, but still matters.

At this point `uflex`-generated lexical analyzers run fast enough for many production purposes, but are not yet fast enough to compete with handwritten lexical analyzers. Unicon's handwritten lexical analyzer in `unicon/uni/unicon/unilex.icn` is written in about 750 lines of Unicon. It is organized into about 39 procedures and uses string scanning. A test program that uses this lexical analyzer to process files named on the command line looks like this:

```
link unilex
procedure main(argv)
    t1 := &time
    count := 0
    yyin := open(argv[1]) | stop("can't open ", argv[1] | "no file given")
    while (i := yylex()) > 0 do {
        count += 1
    }
    close(yyin)
    t2 := &time
```

```
write("processed ", count, " tokens in ", t2-t1, "ms")  
end
```

Linked with the handwritten Unicon lexical analyzer, the above program can be run on large .icn files such as the uflex-generated lexical analyzer in the test suite, uniflex.icn from uniflex.l. At 25KLOC and 869KB this is larger than the largest handwritten Unicon source example, the 9.8KLOC, 272KB file tests/graphics/gpxtest.icn, which is a fatter, self-contained version of ipl/gprogs/gpxtest.icn. In a basic test (./unilex uniflex.icn), the handwritten lexical analyzer processing 214469 tokens in around of 0.53 seconds on an i7-5700 running Zorin 18 Linux. The uflex-generated lexical analyzer runs on the same input in 9.8 seconds, roughly 18x slower than the handwritten lexical analyzer.

As it stands currently, uflex generates a lexical analyzer that produces the same output but is an order of magnitude slower. uflex is ready to be taken seriously, but further opportunities for improvement abound. Its generated deterministic finite automata may benefit from further performance tuning; for example it is possible to generate a better representation of the finite automaton.

6 6. Conclusions

Despite Unicon's plethora of string processing facilities, a yacc-compatible lexical analyzer generator remains of high value to language developers. With Uflex, it now becomes more practical to write tools such as compilers and transpilers in Unicon for full-sized languages.

Uflex may be useful in language processing either as a standalone application or, as is more usual, in conjunction with a parser. Uflex is still immature but is now useful for experimental compiler prototyping.

6.0.1 Acknowledgements

Katrina Ray wrote a prototype lexical analyzer generator for Icon/Unicon called ulex. Uflex began with a preprocessor for ulex (ulpp) that added lex macros. This was followed by a full translation of ulex from C to Unicon. The result generated lexical analyzers that were too slow to be useful. A conversion from NFA to DFA was added by Susan Jeffery. The code was

further debugged and much extended in order to support many toy lexical analyzers produced for use in Clinton Jeffery’s compiler construction courses.

Katrina Ray’s original development of `ulex` was supported by an NMSU fellowship and by the National Library of Medicine during its development, and we thank Katrina and the NLM for this assistance. Ray Pereda wrote the first version of `UTR#2` before any working Icon/Unicon lexical analyzer generator existed, and we thank him for his contributions, including the awesome `iyacc` tool. The `UTR2` text was substantially altered and eventually rewritten from Ray’s document to describe `ulex` and eventually `uflex`.

7 References

- Jeffery03 C. Jeffery, S. Mohamed, R. Pereda, and R. Parlett. Programming with Unicon. <http://unicon.sf.net/book/ub.pdf>, 2003.
- Lesk75 M.E. Lesk and E. Schmidt. LEX – Lexical Analyzer Generator . Computer Science Technical Report No. 39, Bell Laboratories, Murray Hill New Jersey, October 1975.
- Levine09 J.R. Levine. `flex & bison` , O’Reilly & Associates, Cambridge, Massachusetts, 2009.
- Pereda18 Ray Pereda. `Iyacc – a Parser Generator for Icon` , Unicon Technical Report 3a, <http://unicon.org/utr/utr3.pdf>, February 2018.
- Jeffery24 Clinton Jeffery. Build Your Own Programming Language, second edition. Packt, Birmingham UK, 2024.

7.0.1 Appendix: Differences Between `Uflex` and UNIX `lex(1)` and `flex(1)`

This appendix summarizes the known differences between `Uflex` and other implementations of the UNIX `lex(1)` tool. `Uflex` still has several important limitations to note.

- Naïve semantic actions – semantic actions must be enclosed in curly brackets

In addition, the following (Flex) operators are not yet implemented.

(?r:patt)

interpret regex patt with option r (i, s, or x)

(?-r:patt) (?r-s:patt)

more options formats

r/s

lookahead operator

^r

beginning of line context

r\$

end of line context

r{2,5}

ranges

r{4}

fixed # of repetitions

r{2,}

N-or-more r's construct

<s>r

start conditions

<s1,s2,s3>r

multiple start conditions

<*>r

any start condition

«EOF»

end-of-file

<s1,s2>«EOF»

end-of-file occurring in start conditions